

what does serde do in rust and toml

Serde is the main framework in Rust used to [convert code data](#) into text or bytes (serialization) and back (deserialization). In this ecosystem, TOML acts as a specific text file format (often used for config files) that Serde can read and write, while Cargo.toml is the configuration file where you declare Serde as a project dependency. [Serde v4](#)

What Serde Does in Rust

- **Translates data:** Changes Rust data types (like structs and enums) into formats like JSON or TOML, and turns those formats back into native Rust data. [Serde v4](#)
- **Automates code:** Uses simple macros (`#[derive(Serialize, Deserialize)]`) so you do not have to write manual conversion code for every field. [Serde v4](#)
- **Stays flexible:** Keeps the core rules separate from specific data formats, meaning your data structures work the same way regardless of the file type you choose. [Rustify](#)

How Serde Works with TOML

- **Reads config files:** Uses a companion crate (like `toml`) alongside core Serde to parse a `toml` configuration file directly into a strongly typed Rust struct.
- **Saves settings:** Takes a Rust program's current state or settings struct and writes it back out as formatted TOML text.
- **Configured in Cargo.toml:** You enable Serde by writing required entries and features (like `derive`) inside your project's Cargo.toml manifest file. [GitHub Pages document... v4](#)

If you'd like, let me know:

- Do you want to see a **code example** of reading a TOML file into Rust?
- Are you trying to add Serde to your **Cargo.toml** right now?

[📄](#) [↩](#) [🔗](#) [🔍](#) [⋮](#)

give simple example

To use Serde with TOML, you define a Rust struct that matches your data structure and use Serde attributes to handle the conversion.

1. Update Cargo.toml

Add the [serde](#) and [toml](#) crates to your dependencies. You must enable the `derive` feature for Serde.

```
toml

[dependencies]
serde = { version = "1.0", features = ["derive"] }
toml = "0.8"

Use code with caution.
```

2. Write the Rust Code

This example shows how to turn a TOML string into a Rust struct (**deserialization**) and vice versa (**serialization**).

```
rust

use serde::{Deserialize, Serialize};

// 1. Define your struct and derive Serde traits
#[derive(Serialize, Deserialize, Debug)]
struct Config {
    title: String,
    version: u32,
    enabled: bool,
}

fn main() {
    // --- DESERIALIZATION: TOML String -> Rust Struct ---
    let toml_string = r#"
        title = "My Awesome App"
        version = 2
        enabled = true
    "#;

    let decoded: Config = toml::from_str(toml_string).unwrap();
    println!("Decoded Struct: {:?}", decoded);
    println!("Title field: {}\n", decoded.title);

    // --- SERIALIZATION: Rust Struct -> TOML String ---
    let my_config = Config {
        title: String::from("New Export Config"),
        version: 1,
        enabled: false,
    };

    let encoded_toml = toml::to_string(&my_config).unwrap();
    println!("Encoded TOML:\n{}", encoded_toml);
}

Use code with caution.
```

If you want to take this further, let me know:

- Do you want to see how to **read from an actual file** on your disk?
- Do you need to handle **nested tables** or lists in your TOML?

[📄](#) [↩](#) [🔗](#) [🔍](#) [⋮](#)

But I don't understand the need of it. Why do we need to convert TOML string into Rust struct? and vice versa?

We need to convert TOML into a Rust struct [because computer programs](#) and human developers look at data in completely different ways.

Without this conversion, a computer program cannot easily understand or use the settings you type into a text file.

Why Rust Needs the Struct (TOML → Rust)

- **Computer efficiency:** Rust cannot easily read a plain text line like `"version = 2"`. Converting it into a struct changes that text into a type-safe integer (`2`) stored directly in computer memory.
- **Type prevention:** If you misspell a setting name or put text where a number belongs, Rust will catch the error immediately instead of letting your app crash later. [🔗](#)
- **Easy code access:** It lets you use clean code like `config.version` throughout your app, instead of manually searching through text lines every time you need a value.

Why You Need the TOML (Rust → TOML)

- **Human readability:** When your app closes, its memory clears. To save user settings, you must write them to a hard drive as readable text (like TOML) so a human can open and edit them later. [🔗](#)
- **System sharing:** A Rust struct only exists inside a Rust program. Converting it to a standardized format like TOML or JSON lets your app share data with other programs written in Python, JavaScript, or C++.

If you want to see this in action, let me know:

- Would you like to see how **error handling** works when a user types the wrong data type in a TOML file?
- Do you want an example of a **real-world use case**, like managing game settings or database credentials?

[📄](#) [↩](#) [🔗](#) [🔍](#) [⋮](#)

Ok so serde i used to convert toml to rust struct and vice versa, do we do this for other languages too?

Yes, we do this exact same process in almost every programming language.

While the tool is called **Serde** only in Rust, the concept is universal. It is called **Serialization** (saving data to a file) and **Deserialization** (loading a file into code objects). [🔗](#)

How Other Languages Do It

- **JavaScript / TypeScript:** Uses built-in tools like `JSON.stringify()` and `JSON.parse()`. It turns JavaScript objects into text files and back.
- **Python:** Uses built-in modules like `json.dumps()` or `toml.loads()`. These turn text files into Python dictionaries or custom classes.
- **Java / C#:** Uses popular external libraries (like `Jackson` or `Newtonsoft.Json`). They convert text files directly into Java or C# classes, matching fields just like Serde. [🔗](#)

Summary of Terms

No matter the language, the workflow is always the same:

Code Object (Struct/Class) $\xrightleftharpoons[\text{Deserialization (Load)}]{\text{Serialization (Save)}}$ Text File (TOML/JSON/YAML)

If you want to explore further, let me know:

- Do you want to see a quick **Python or JavaScript comparison** to see how it looks outside of Rust?
- Are you curious about why people choose **TOML over JSON** or vice versa?

[📄](#) [↩](#) [🔗](#) [🔍](#) [⋮](#)

So should i consider it a bit downside that in languages like JS, Python we can directly use inbuild methods but in case ... [👇](#)

Not necessarily. While it feels like an extra step, using an external library like Serde in Rust is actually considered a major **architectural advantage** rather than a downside. [🔗](#)

Here is why languages handle this differently, and why Rust's approach is highly valued.

Why JS and Python Have Built-in Tools

- **Dynamic nature:** JS and Python are dynamic languages. They do not need to know the strict "shape" (data type) of your data before the program runs. [🔗](#)
- **Web heritage:** JavaScript was literally invented for the web, so having built-in JSON support was a core requirement from day one. [🔗](#)
- **Convenience first:** These languages trade a bit of execution speed and memory efficiency to give developers maximum convenience out of the box. [🔗](#)

Why Rust and Java Use Libraries Like Serde

- **Lean standard library:** Rust purposely keeps its core standard library incredibly small. This ensures that Rust can run on everything from tiny microcontrollers (embedded systems) to massive servers without carrying useless code weight. [🔗](#)
- **The "Serde Ecosystem" advantage:** In Python, if you want to switch from JSON to TOML, you usually have to learn a completely new library with different functions. In Rust, Serde is a generic framework. You learn `serde` once, and it works identically whether you plug in JSON, TOML, YAML, or binary formats. [🔗](#)
- **Zero-cost abstractions:** Serde does its heavy lifting *while your code is compiling*, not while it is running. It generates highly optimized machine code specifically for your structs, making it vastly faster and safer than the built-in interpreters in JS or Python. [🔗](#)

The Trade-off Summary

- **JS/Python:** High convenience, slower runtime performance, less type safety.
- **Rust/Java:** Small setup step (adding a dependency), blazing-fast runtime performance, total type safety. [🔗](#)

If you'd like to dive deeper, let me know:

- Do you want to see how **fast Serde** is compared to other languages?
- Would you like to see how easy it is to **switch a Rust struct from TOML to JSON** using Serde?

[📄](#) [↩](#) [🔗](#) [🔍](#) [⋮](#)

Both

Here is how Serde stacks up in terms of speed, followed by how easy it is to flip your code from TOML to JSON.

Part 1: How Fast Is Serde? (The Benchmarks)

Serde is drastically faster than the built-in serialization tools in dynamic languages like Python and JavaScript.

Because Serde works during **compilation**, it writes a custom, highly optimized parser just for your specific struct. It doesn't waste time figuring out data shapes while the app is running. [Reddit v4](#)

When parsing large blocks of data (such as 1 GB of data logs), third-party industry benchmarks highlight the massive difference in execution speed: [Scannerdev](#)

- **Python (`json` module):** Takes roughly **8 to 12 seconds**. It is slow because it checks types line-by-line at runtime.
- **JavaScript (`JSON.parse`):** Takes roughly **6 to 8 seconds**. It is heavily optimized by browser engines but bound by JS's dynamic nature.
- **Rust (`serde`):** Takes roughly **2 seconds**. It handles the same data **4x to 6x faster** than the dynamic options because it allocates minimal memory and translates directly to native machine code. [Reddit v2](#)

Part 2: Switching Formats Instantly

Because Serde separates your Rust data structures from the final output format, you do not need to rewrite your code if your project specifications change. You can switch text formats by changing just a few lines.

Step 1: Update your Cargo.toml

Simply include the [serde_json](#) crate alongside your existing TOML setup:

```
toml

[dependencies]
serde = { version = "1.0", features = ["derive"] }
toml = "0.8"
serde_json = "1.0" # <- Just add this crate

Use code with caution.
```

Step 2: Swap the methods in your Code

Your `Config` struct stays exactly the same. You only swap out the converter function:

```
rust

use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize, Debug)]
struct Config {
    title: String,
    version: u32,
    enabled: bool,
}

fn main() {
    // 1. Working with TOML
    let toml_string = "title = 'App'\nversion = 1\nenabled = true";
    let from_toml: Config = toml::from_str(toml_string).unwrap();

    // 2. Switching to JSON takes exactly one line of change!
    let json_string = r#"{"title": "App", "version": 1, "enabled": true}"#;
    let from_json: Config = serde_json::from_str(json_string).unwrap();

    // 3. Export back out to JSON instead of TOML
    let exported_json =
        serde_json::to_string_pretty(&from_json).unwrap();
    println!("{}", exported_json);
}

Use code with caution.
```

If you'd like to continue, let me know:

- Do you want to see how to handle data fields where the name changes (e.g., `camelCase` in JSON but `snake_case` in TOML)?
- Would you like to look at how Serde safely handles **missing fields** using default values?

[📄](#) [↩](#) [🔗](#) [🔍](#) [⋮](#)

🔗🔗🔗 3 sites

Overview - Serde

Serde also provides a derive macro to generate serialization implementations for...

Serde

Rust - Converting between file formats - JSON, YAML, & TOML

March 26, 2020 5-minute read. Rust's serde library is a generic serialization/deserialization framework that has been...

GitHub Pages documentation

Serde in Rust - liveBook - Manning

Serde in Web Development Serde is particularly useful in the context of web development, where it is often used to...

liveBook - Manning

Show all

🔗🔗🔗 3 sites

Mastering Option and Result in Rust | by Enzo Lombardi |

29 Dec 2023 — The commented-out line illustrates a key point. In languages with ru...

Medium

Parse and Generate TOML in Rust | Parse and Generate Formats In...

Generating TOML formatted strings from Rust data structures is straightforward us...

SSOJet

liveBook - Manning

Overview of TOML (Tom's Obvious, Minimal Language) TOML (Tom's Obvious, Minimal Language), which stands...

liveBook - Manning

🔗🔗🔗 7 sites

Serde alternatives - The Rust Programming Language Forum

1 Dec 2024 — and it looks like the answer ATM is there really isn't any practical alternative to serde, as a universal...

The Rust Programming Language ...

Exploring Rust For Data Engineering Part 1

20 Jun 2023 — Rust has a framework called Serde which can serialize and deserialize various data formats to and from...

kurtelhauser.com

Convert struct to byte array and read this via implementing "std::io::Read" trait : rlearnrust

12 Dec 2020 — Serialization is the process of taking some data stored in memory and saving it to a file on disk, and...

Reddit

Show all

🔗🔗🔗🔗 26 sites

Is rust serde unique? : r/rust

20 Apr 2023 — You say a "consistent pattern shared between all libraries", but there is...

Reddit

JSON and Rust: Why `serde_json` is the top choice

21 Sept 2020 — While there are plenty of other JSON (JSON object) crates availabl...

LogRocket Blog

What Makes Python Best Programming Language For...

6 Aug 2022 — Let's compare it (Python Programming) with JavaScript, because...

DEV Community

Show all

🔗🔗🔗 4 sites

As a programming language, how does Rust compare to Python? : r/rust

19 Mar 2020 — Here's some more information about Rust and Python: **Rust** Rust is a strongly typed language. It...

Reddit

Serverless Speed: Rust vs. Go, Java, and Python in AWS ...

11 Sept 2025 — The high-level takeaways. The chart above shows the performance...

Scannerdev

Rust Vs. Other Programming Languages: What Sets Rust Apart?

13 Jun 2024 — When is Rust Better Than JavaScript? * Memory Management One k...

Strapi

Show all